PDFly Chat: LangChain Empowered PDF Assistant

Pravallika Mummadi, Hemanth Sai Kagitha, Sri Venkata Satya Sai Charan Teja Tallam

Northwest Missouri State University, Maryville MO 64468, USA
S555592@nwmissouri.edu, S559891@nwmissouri.edu, S559495@nwmissouri.edu

1 Team Name

PDF Insight Clan

2 Team Members

- Pravallika Mummadi
- Hemanth Sai Kagitha
- Sri Venkata Satya Sai Charan Teja Tallam

3 Project Title

PDFly Chat: LangChain Empowered PDF Assistant

4 Project Idea

Our project aims to revolutionize document interaction through the development of a LangChain-powered PDF chatbot. By integrating advanced generative AI and contextual understanding, users can engage in natural language conversations to extract, summarize, and interact with PDF content effortlessly. Our chatbot will provide seamless navigation, enabling users to ask questions, retrieve key information, and annotate documents with ease. Through overcoming the limitations of traditional LLMs, we aim to empower users across various domains to harness the full potential of conversational AI within the realm of PDF documents.

5 Tools & Technologies used

- **Langchain (0.0.154):** A Python library designed for building applications with language models like GPT-3 and GPT-4. It facilitates the integration of language AI features into the system for text generation and processing.

- **PyPDF2 (3.0.1):** This library is used for manipulating PDF files in Python. It will be utilized to read content from PDF files, extract text data, and perform other PDF-related tasks such as splitting and merging documents.

- **Python-dotenv (1.0.0):** This tool manages environment variables in the development process, aiding in the secure loading of API keys and sensitive information from an .env file.

- **Streamlit (1.18.1):** A framework for building web applications in Python. It will be used to create an interface for user interaction with the knowledge database.

- **FAISS-cpu (1.7.4):** A library for efficient similarity search and clustering of dense vectors. It will handle the embedding and indexing of text chunks for semantic search within the application.

- **Streamlit-extras:** A package providing additional components for Streamlit applications. It extends the functionality of the user interface with custom components.

- **Altair (¡5):** A statistical visualization library for Python. Altair will be used for visualizing data in the Streamlit application, such as search result statistics or embedding relationships.

- **TikToken (0.6.0):** A utility for handling tokens or text processing. It may assist in text tokenization or managing API tokens for services like OpenAI.

6 Chat-bot Architecture

This flowchart depicts an information retrieval system that extracts the content of a PDF file and divides it into smaller text segments. These segments are then transformed into numerical vectors known as embeddings, which capture their semantic meaning. These embeddings are indexed in a knowledge database for efficient retrieval. When a user asks a question, it is also converted into an embedding. A semantic search is conducted against the index to identify the most relevant information. The results are then ranked, and a Large Language Model (LLM) Generative AI, such as GPT-4, utilizes this data to generate a response to the user's inquiry.

6.1 Steps Involved

1. **PDF File**: The process begins with a PDF document that contains the data or context of interest.
2. **Extract data/Context**: The content of the PDF is extracted to get the textual data.

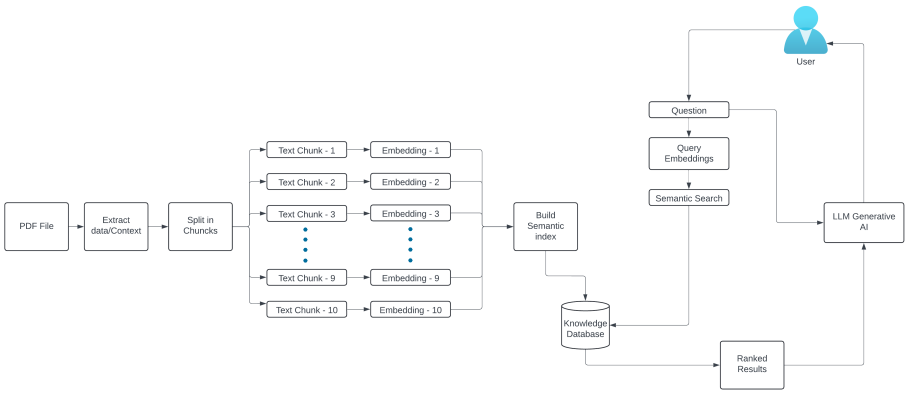

Fig. 1. Architecture Diagram

3. **Split in Chunks**: The extracted text is then split into smaller pieces, referred to as chunks.
4. **Text Chunk - 1 to Text Chunk - 10**: These are the individual text segments created from the original document.
5. **Embedding - 1 to Embedding - 10**: Each text chunk is converted into an embedding.
6. **Build Semantic Index**: The embeddings are used to create a semantic index.
7. **Knowledge Database**: This is the storage where the indexed data is kept.
8. **User**: This is the person interacting with the system.
9. **Question**: The user poses a question to the system.
10. **Query Embeddings**: The question is converted into an embedding.
11. **Semantic Search**: The query embedding is used to perform a semantic search.
12. **Ranked Results**: The results of the semantic search are ranked.
13. **LLM Generative AI**: This is the AI model that generates responses.
14. **User (feedback loop)**: The results or information provided by the LLM are delivered back to the user.

7 Environment Setup

- **Install Python**: Ensure that Python 3.6 or newer is installed on your machine.
- **Set up a Virtual Environment** (optional but recommended):
  - Create a virtual environment: `python -m venv venv`
  - Activate the virtual environment:
    - On Windows: `venv\Scripts\activate`
    - On Unix or MacOS: `source venv/bin/activate`

– **Install Required Libraries**: Execute the following command to install the necessary Python packages:

```
pip install langchain PyPDF2 python-dotenv streamlit faiss-cpu altair openai
```

If you're using the `streamlit-extras` package, make sure to install it as well from the source it's available from.

7.1 Environment Variables

– Create a `.env` file in the root of the project directory.
– Place your OpenAI API key in the `.env` file:

```
OPENAI_API_KEY=your_api_key_here
```

7.2 Obtaining API Keys

Sign up for an account at OpenAI and Hugging Face to obtain API keys for access to language models.

8 Application Code Breakdown

8.1 Import Libraries

Import all required Python libraries that are needed to run the application.

8.2 Sidebar Information

Use Streamlit to create a sidebar in the application UI to display information about the application.

8.3 File Upload

– Use Streamlit's `file_uploader` function to allow users to upload a PDF file.
– Use PyPDF2 to read the uploaded PDF and extract text.

8.4 Text Splitting

Use `RecursiveCharacterTextSplitter` from LangChain to split the extracted text into manageable chunks.

8.5 Embedding Creation

– Use `HuggingFaceEmbeddings` to create embeddings for each text chunk.
– Store embeddings in a FAISS index for efficient similarity search. This index is saved as a `.pkl` file for persistence.

8.6 User Query

Provide a text input field using Streamlit for users to enter their queries.

Search and Response

- Perform a similarity search on the FAISS index using the user's query.
- Retrieve and rank the most relevant chunks from the PDF.
- Use a pre-trained Hugging Face model to generate a response based on the relevant chunks and user query.

8.7 Display Results

Use Streamlit to display the generated response to the user's query.

9 Running the Application

Start the Streamlit Server

Execute `streamlit run your_script.py` in the terminal where `your_script.py` is the name of your Python script.

9.1 Interacting with the Application

- The application should now be running in a web browser.
- Upload a PDF file and ask questions to interact with the chatbot.

10 Troubleshooting

- **Debugging**: Check the console for any error messages that may help identify issues in the code or with dependencies.
- **API Key Security**: Ensure that the API keys are not hardcoded in the script and are securely stored in the `.env` file.
- **Library Versions**: Make sure all libraries are compatible with each other. If you encounter issues, try installing specific versions that are known to work together.

By following these detailed steps, anyone should be able to successfully set up and run the project without encountering significant difficulties. This guide assumes a basic understanding of Python programming and command-line operations.

11 Conclusion

The PDF Chatbot project integrates Streamlit for the user interface, LangChain for text processing tasks, and Hugging Face for accessing advanced language models, culminating in an interactive tool for querying PDF documents. Users can effortlessly upload PDFs, which are then processed into smaller text chunks for analysis. Leveraging LangChain's embedding capabilities, the application generates embeddings for these chunks, facilitating semantic analysis and similarity search. Through a chat interface, users can pose questions about the document content, and the application utilizes Hugging Face's Flan T5 models for question answering, providing relevant responses. Furthermore, the chat history feature enables users to review past interactions. In conclusion, the PDF Chatbot project showcases the seamless integration of state-of-the-art technologies to create a user-friendly tool for extracting information from PDF documents, with potential applications in research, education, and professional settings.

12 Future Work

For future work, several avenues could be explored to enhance the functionality and utility of the PDF Chatbot. Firstly, improving the robustness and accuracy of the question-answering system by fine-tuning the language model on domain-specific data could lead to more accurate responses tailored to the content of the PDF documents. Additionally, incorporating natural language understanding (NLU) techniques such as named entity recognition (NER) and sentiment analysis could provide deeper insights into the document content and user queries.

Furthermore, implementing a more interactive and intuitive user interface with features such as voice input/output, multi-document support, and real-time collaboration capabilities would enhance the user experience and broaden the application's appeal. Integration with cloud storage platforms like Google Drive or Dropbox could enable seamless access to PDF documents stored online, increasing accessibility and convenience for users.

Moreover, exploring additional functionalities such as document summarization, topic modeling, and document clustering could provide users with more comprehensive insights and facilitate knowledge discovery from large volumes of PDF documents. Finally, conducting user studies and gathering feedback to iteratively improve the application based on user preferences and requirements would be essential for ensuring its effectiveness and usability in real-world scenarios.

13 Citations

– Github